📘

The Complete SQL Query Book

200 interview questions, explained in plain English — with real output tables for every single query.

A beginner-friendly companion to the SQL Interview Preparation cheat sheet. Every original query is preserved exactly as given, with an added explanation and a real, executed result table so you can see — not just read — what each query produces.

The Sample Database

Every query in this book runs against this same small company database — an HR + e-commerce system with 7 tables. Get familiar with these tables first; once you know what data lives where, every query below will make a lot more sense.

employees · 15 rows

Columns: `emp_id`, `name`, `age`, `gender`, `salary`, `department`, `designation`, `manager_id`, `hire_date`, `city`, `bonus`, `experience_years`

The heart of this book's examples — 15 employees across IT, HR, Finance, and Marketing, each with a salary, city, manager, and hire date. Most queries in Levels 1–6 use this table.

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mum
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi
3	Rahul Singh	45	M	85,000	Finance	Finance Head	1	2018-07-10	Mum
4	Neha Gupta	29	F	72,000	IT	Developer	1	2021-02-01	Bang

4 rows returned.

departments · 5 rows

Columns: `dept_id`, `dept_name`, `location`, `budget`

5 departments with their location and yearly budget. Used to practice JOINs against `employees` via the shared `department` / `dept_name` columns.

dept_id	dept_name	location	budget
1	IT	Mumbai	5,000,000

dept_id	dept_name	location	budget
2	HR	Delhi	2,000,000
3	Finance	Mumbai	3,000,000
4	Marketing	Bangalore	2,500,000

4 rows returned.

projects · 6 rows

Columns: `project_id`, `project_name`, `department`, `status`, `start_date`, `end_date`, `budget`

6 company projects, each tied to a department, with a status (Active/Completed/Planning) and budget.

project_id	project_name	department	status	start_date	end_date	budget
1	AI Implementation	IT	Active	2024-01-01	2024-12-31	1,500,000
2	Employee Portal	IT	Completed	2023-06-01	2023-12-31	800,000
3	Financial Audit	Finance	Active	2024-02-01	2024-08-31	500,000
4	Market Research	Marketing	Planning	2024-03-01	2024-09-30	300,000

4 rows returned.

customers · 10 rows

Columns: `customer_id`, `name`, `city`, `registration_date`, `email`

10 companies who place orders. Used together with `orders` to practice JOINs, subqueries, and customer-analysis queries.

customer_id	name	city	registration_date	email
1	TechCorp	Mumbai	2023-01-10	tech@corp.com
2	GlobalSolutions	Delhi	2023-02-15	info@global.com
3	InnovationLabs	Bangalore	2023-03-20	contact@innovate.com
4	AlphaInc	Mumbai	2023-04-25	alpha@inc.com

4 rows returned.

orders · 15 rows

Columns: `order_id`, `customer_id`, `order_date`, `amount`, `status`, `product_category`

15 customer orders with amount, status, and category. This is the main table for aggregate and revenue-analysis queries.

order_id	customer_id	order_date	amount	status	product_category
101	1	2024-01-10	15,000	Completed	Electronics
102	1	2024-01-15	23,000	Pending	Software
103	2	2024-01-20	12,000	Completed	Hardware
104	3	2024-02-01	50,000	Completed	Electronics

4 rows returned.

products · 10 rows

Columns: `product_id`, `product_name`, `category`, `price`, `stock_quantity`

10 products for sale, each with a category, price, and stock level.

product_id	product_name	category	price	stock_quantity
1	Laptop Pro	Electronics	75,000	50
2	Wireless Mouse	Accessories	1,500	200
3	27" Monitor	Electronics	25,000	30
4	Mechanical Keyboard	Accessories	5,000	100

4 rows returned.

order_items · 15 rows

Columns: `order_item_id`, `order_id`, `product_id`, `quantity`, `price`

The line items inside each order (which product, how many, at what price). Connects `orders` to `products` for detailed sales analysis.

order_item_id	order_id	product_id	quantity	price
1	101	1	2	75,000
2	101	2	1	1,500
3	102	5	1	100,000
4	103	3	1	25,000

4 rows returned.

Level 1: Basics

Learn the four foundational building blocks of every query: choosing columns with SELECT, filtering rows with WHERE, sorting with ORDER BY, and controlling result size with LIMIT/OFFSET. Nearly every later query in this book builds on these basics.

Q1. Display all employees EASY

```
SELECT * FROM employees;
```

WHAT THIS QUERY DOES

The SELECT * pattern means 'give me every column.' Combined with FROM employees and no WHERE filter, this simply dumps every row of the table exactly as it's stored — the SQL equivalent of opening a spreadsheet and looking at everything. It's usually the very first query anyone runs against a new table, just to see what's in it.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	1
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi	8,
3	Rahul Singh	45	M	85,000	Finance	Finance Head	1	2018-07-10	Mumbai	1
4	Neha Gupta	29	F	72,000	IT	Developer	1	2021-02-01	Bangalore	9,
5	Vikram Reddy	38	M	90,000	IT	Team Lead	1	2017-11-25	Delhi	1
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5,
7	Karthik Nair	51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	2
8	Meera Krishnan	33	F	68,000	Marketing	Marketing Lead	5	2020-08-12	Delhi	7,
9	Arjun Reddy	42	M	82,000	IT	Senior Developer	5	2016-12-01	Mumbai	1
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4,
11	Rajesh Kumar	39	M	70,000	Finance	Financial Analyst	3	2019-06-10	Chennai	6,
12	Deepa Iyer	34	F	64,000	HR	HR Specialist	2	2020-11-15	Hyderabad	5,

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
13	Suresh Babu	47	M	88,000	IT	Solution Architect	5	2014-03-01	Chennai	18
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5,
15	Ravi Teja	44	M	91,000	IT	Technical Manager	1	2016-07-15	Mumbai	1(

15 rows returned.

Q2. Display employee names only EASY

```
SELECT name FROM employees;
```

WHAT THIS QUERY DOES

Instead of *, we name the exact column we want: name. SQL only returns the columns you list, in the order you list them — so this returns a single-column result with 15 rows, one name per employee, ignoring salary, age, and everything else.

RESULT ON OUR SAMPLE DATA

name
Aarav Sharma
Priya Patel
Rahul Singh
Neha Gupta
Vikram Reddy
Ananya Menon
Karthik Nair
Meera Krishnan
Arjun Reddy
Sneha Patel
Rajesh Kumar
Deepa Iyer
Suresh Babu
Kavya Nair
Ravi Teja

15 rows returned.

Q3. Display unique departments EASY

```
SELECT DISTINCT department FROM employees;
```

WHAT THIS QUERY DOES

`DISTINCT` removes duplicate values from the result. There are 15 employees but only 4 departments, so without `DISTINCT` you'd see 'IT' repeated many times; with it, each department name appears exactly once.

RESULT ON OUR SAMPLE DATA

department
IT
HR
Finance
Marketing

4 rows returned.

Q4. Employees with salary > 50000 EASY

```
SELECT * FROM employees WHERE salary > 50000;
```

WHAT THIS QUERY DOES

`WHERE` filters rows — only rows where the condition is true make it into the result. Here every employee happens to earn more than 50,000, so all 15 rows pass the filter; change the number and you'd see fewer rows come back.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	1
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi	8
3	Rahul Singh	45	M	85,000	Finance	Finance Head	1	2018-07-10	Mumbai	1
4	Neha Gupta	29	F	72,000	IT	Developer	1	2021-02-01	Bangalore	9
5	Vikram Reddy	38	M	90,000	IT	Team Lead	1	2017-11-25	Delhi	1
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5
7		51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	2

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
	Karthik Nair									
8	Meera Krishnan	33	F	68,000	Marketing	Marketing Lead	5	2020-08-12	Delhi	7
9	Arjun Reddy	42	M	82,000	IT	Senior Developer	5	2016-12-01	Mumbai	1
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4
11	Rajesh Kumar	39	M	70,000	Finance	Financial Analyst	3	2019-06-10	Chennai	6
12	Deepa Iyer	34	F	64,000	HR	HR Specialist	2	2020-11-15	Hyderabad	5
13	Suresh Babu	47	M	88,000	IT	Solution Architect	5	2014-03-01	Chennai	1
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5
15	Ravi Teja	44	M	91,000	IT	Technical Manager	1	2016-07-15	Mumbai	1

15 rows returned.

Q5. Salary between 40000 and 80000 EASY

```
SELECT * FROM employees WHERE salary BETWEEN 40000 AND 80000;
```

WHAT THIS QUERY DOES

`BETWEEN a AND b` is shorthand for `salary >= a AND salary <= b` — both endpoints are included. It reads almost like plain English, which is why it's preferred over writing the two comparisons out separately.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	1
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi	8
4	Neha Gupta	29	F	72,000	IT	Developer	1	2021-02-01	Bangalore	9
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5
8	Meera Krishnan	33	F	68,000	Marketing	Marketing Lead	5	2020-08-12	Delhi	7
10		26	F	58,000	Marketing		8	2023-01-20	Bangalore	4

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
	Sneha Patel					Marketing Executive				
11	Rajesh Kumar	39	M	70,000	Finance	Financial Analyst	3	2019-06-10	Chennai	6,
12	Deepa Iyer	34	F	64,000	HR	HR Specialist	2	2020-11-15	Hyderabad	5,
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5,

9 rows returned.

Q6. Employees hired after 2020 EASY

```
SELECT * FROM employees WHERE hire_date > '2020-12-31';
```

WHAT THIS QUERY DOES

Dates in SQL can be compared just like numbers, as long as they're stored in a consistent format (here, `YYYY-MM-DD` text, which sorts correctly as a string too). 'After 2020' is expressed as 'after the last day of 2020' rather than trying to compare a date to a bare year.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bo
4	Neha Gupta	29	F	72,000	IT	Developer	1	2021-02-01	Bangalore	9,0
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5,0
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4,0
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5,0

4 rows returned.

Q7. Employees whose name starts with A EASY

```
SELECT * FROM employees WHERE name LIKE 'A%';
```

WHAT THIS QUERY DOES

`LIKE` does pattern matching on text. The `%` symbol is a wildcard meaning 'any sequence of characters (including none)' — so `'A%'` matches any name that starts with a capital A, followed by anything at all.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bonu
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	10,00
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5,000
9	Arjun Reddy	42	M	82,000	IT	Senior Developer	5	2016-12-01	Mumbai	11,00

3 rows returned.

Q8. Ends with n EASY

```
SELECT * FROM employees WHERE name LIKE '%n';
```

WHAT THIS QUERY DOES

Same wildcard idea as the previous query, but now the % is at the *front*: 'anything, followed by a literal n' — meaning the name must end in the letter n. Placement of % is what controls whether you're matching a prefix, suffix, or something in the middle.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bonu
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5,00
8	Meera Krishnan	33	F	68,000	Marketing	Marketing Lead	5	2020-08-12	Delhi	7,00

2 rows returned.

Q9. Contains "ar" EASY

```
SELECT * FROM employees WHERE name LIKE '%ar%';
```

WHAT THIS QUERY DOES

Wildcards on *both* sides mean 'anything, then the letters ar, then anything' — in other words, match if 'ar' shows up anywhere in the name at all, regardless of position.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bon
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	10,0

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bon
7	Karthik Nair	51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	20,(
9	Arjun Reddy	42	M	82,000	IT	Senior Developer	5	2016-12-01	Mumbai	11,(
11	Rajesh Kumar	39	M	70,000	Finance	Financial Analyst	3	2019-06-10	Chennai	6,0(

4 rows returned.

Q10. NULL manager EASY

```
SELECT * FROM employees WHERE manager_id IS NULL;
```

WHAT THIS QUERY DOES

NULL means 'no value' — it's SQL's way of representing missing or not-applicable data. Critically, you can never write `manager_id = NULL` (that never matches anything, even other NULLs); you must use the special `IS NULL` check instead. This finds the one employee with no manager — presumably the top of the org chart.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bonu
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	10,00

1 row returned.

Q11. Salary ascending EASY

```
SELECT * FROM employees ORDER BY salary ASC;
```

WHAT THIS QUERY DOES

`ORDER BY` sorts the result set. `ASC` (ascending, the default if you omit it) means lowest values first — so this lists employees starting with the smallest salary and working up to the largest.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4,
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5,
12		34	F	64,000	HR	HR Specialist	2	2020-11-15	Hyderabad	5,

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
	Deepa Iyer									
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi	8,
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5,
8	Meera Krishnan	33	F	68,000	Marketing	Marketing Lead	5	2020-08-12	Delhi	7,
11	Rajesh Kumar	39	M	70,000	Finance	Financial Analyst	3	2019-06-10	Chennai	6,
4	Neha Gupta	29	F	72,000	IT	Developer	1	2021-02-01	Bangalore	9,
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	1
9	Arjun Reddy	42	M	82,000	IT	Senior Developer	5	2016-12-01	Mumbai	1
3	Rahul Singh	45	M	85,000	Finance	Finance Head	1	2018-07-10	Mumbai	1
13	Suresh Babu	47	M	88,000	IT	Solution Architect	5	2014-03-01	Chennai	1
5	Vikram Reddy	38	M	90,000	IT	Team Lead	1	2017-11-25	Delhi	1
15	Ravi Teja	44	M	91,000	IT	Technical Manager	1	2016-07-15	Mumbai	1
7	Karthik Nair	51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	2

15 rows returned.

Q12. Salary descending EASY

```
SELECT * FROM employees ORDER BY salary DESC;
```

WHAT THIS QUERY DOES

Flipping to DESC (descending) reverses the sort direction — highest salary first. This is the more common choice in real reports, since people usually want to see 'the biggest/best/most' items at the top.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
7	Karthik Nair	51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	2
15		44	M	91,000	IT		1	2016-07-15	Mumbai	1

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
	Ravi Teja					Technical Manager				
5	Vikram Reddy	38	M	90,000	IT	Team Lead	1	2017-11-25	Delhi	1
13	Suresh Babu	47	M	88,000	IT	Solution Architect	5	2014-03-01	Chennai	1
3	Rahul Singh	45	M	85,000	Finance	Finance Head	1	2018-07-10	Mumbai	1
9	Arjun Reddy	42	M	82,000	IT	Senior Developer	5	2016-12-01	Mumbai	1
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	1
4	Neha Gupta	29	F	72,000	IT	Developer	1	2021-02-01	Bangalore	9
11	Rajesh Kumar	39	M	70,000	Finance	Financial Analyst	3	2019-06-10	Chennai	6
8	Meera Krishnan	33	F	68,000	Marketing	Marketing Lead	5	2020-08-12	Delhi	7
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi	8
12	Deepa Iyer	34	F	64,000	HR	HR Specialist	2	2020-11-15	Hyderabad	5
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4

15 rows returned.

Q13. Department then salary EASY

```
SELECT * FROM employees ORDER BY department, salary DESC;
```

WHAT THIS QUERY DOES

You can sort by multiple columns at once. This groups all rows with the same department together (alphabetically, since no direction was given for it — so it defaults to ascending), and *within* each department group, sorts by salary from highest to lowest. Think of the second column as a tiebreaker for the first.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	b
3	Rahul Singh	45	M	85,000	Finance	Finance Head	1	2018-07-10	Mumbai	15
11	Rajesh Kumar	39	M	70,000	Finance	Financial Analyst	3	2019-06-10	Chennai	6,
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5,
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi	8,
12	Deepa Iyer	34	F	64,000	HR	HR Specialist	2	2020-11-15	Hyderabad	5,
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5,
7	Karthik Nair	51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	2(
15	Ravi Teja	44	M	91,000	IT	Technical Manager	1	2016-07-15	Mumbai	1(
5	Vikram Reddy	38	M	90,000	IT	Team Lead	1	2017-11-25	Delhi	1:
13	Suresh Babu	47	M	88,000	IT	Solution Architect	5	2014-03-01	Chennai	18
9	Arjun Reddy	42	M	82,000	IT	Senior Developer	5	2016-12-01	Mumbai	1:
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	1(
4	Neha Gupta	29	F	72,000	IT	Developer	1	2021-02-01	Bangalore	9,
8	Meera Krishnan	33	F	68,000	Marketing	Marketing Lead	5	2020-08-12	Delhi	7,
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4,

15 rows returned.

Q14. Top 5 salaries EASY

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 5;
```

WHAT THIS QUERY DOES

LIMIT caps how many rows come back. Sorting highest-first and then taking only the first 5 rows is the standard way to answer 'who are the top N by some measure' — sort, then slice.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bon
7	Karthik Nair	51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	20,0
15	Ravi Teja	44	M	91,000	IT	Technical Manager	1	2016-07-15	Mumbai	16,0
5	Vikram Reddy	38	M	90,000	IT	Team Lead	1	2017-11-25	Delhi	12,0
13	Suresh Babu	47	M	88,000	IT	Solution Architect	5	2014-03-01	Chennai	18,0
3	Rahul Singh	45	M	85,000	Finance	Finance Head	1	2018-07-10	Mumbai	15,0

5 rows returned.

Q15. Lowest 5 salaries EASY

```
SELECT * FROM employees ORDER BY salary ASC LIMIT 5;
```

WHAT THIS QUERY DOES

Same 'sort then slice' idea as the previous query, just sorting the other direction first — ascending order puts the smallest salaries at the top, and `LIMIT 5` grabs just those.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bo
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4,0
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5,0
12	Deepa Iyer	34	F	64,000	HR	HR Specialist	2	2020-11-15	Hyderabad	5,5
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi	8,0
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5,0

5 rows returned.

Q16. Distinct cities EASY

```
SELECT DISTINCT city FROM employees;
```

WHAT THIS QUERY DOES

Same DISTINCT technique as Q3, applied to the city column instead of department — it collapses the 15 employee rows down to just the unique city names among them.

RESULT ON OUR SAMPLE DATA

city
Mumbai
Delhi
Bangalore
Chennai
Hyderabad

5 rows returned.

Q17. Distinct job roles EASY

```
SELECT DISTINCT designation FROM employees;
```

WHAT THIS QUERY DOES

Again, DISTINCT on a different column (designation, i.e. job title) — useful for quickly seeing 'what roles exist in this company' without scrolling through every employee.

RESULT ON OUR SAMPLE DATA

designation
Senior Developer
HR Manager
Finance Head
Developer
Team Lead
HR Executive
Architect
Marketing Lead
Marketing Executive
Financial Analyst
HR Specialist
Solution Architect
Accountant
Technical Manager

14 rows returned.

Q18. Count distinct departments EASY

```
SELECT COUNT(DISTINCT department) FROM employees;
```

WHAT THIS QUERY DOES

This combines two ideas: `DISTINCT` narrows the department column down to unique values, and `COUNT()` (an aggregate function, covered properly in Level 2) then counts how many there are — turning a list of unique departments into a single number.

RESULT ON OUR SAMPLE DATA

COUNT(DISTINCT department)
4

1 row returned.

Q19. Distinct managers EASY

```
SELECT DISTINCT manager_id FROM employees WHERE manager_id IS NOT NULL;
```

WHAT THIS QUERY DOES

`IS NOT NULL` is the opposite of Q10's `IS NULL` — it keeps only rows that *do* have a value. Combined with `DISTINCT`, this lists every unique employee ID that manages at least one other person, excluding the placeholder NULL that would otherwise show up for the top-level employee.

RESULT ON OUR SAMPLE DATA

manager_id
1
2
5
8
3

5 rows returned.

Q20. Distinct designations EASY

```
SELECT DISTINCT designation FROM employees;
```

WHAT THIS QUERY DOES

Identical to Q17 — included here again in the original cheat sheet as a reminder that DISTINCT-on-a-column is a pattern worth drilling until it's automatic, not just a one-off trick.

RESULT ON OUR SAMPLE DATA

designation
Senior Developer
HR Manager
Finance Head
Developer
Team Lead
HR Executive
Architect
Marketing Lead
Marketing Executive
Financial Analyst
HR Specialist
Solution Architect
Accountant
Technical Manager

14 rows returned.

Q21. First 10 employees EASY

```
SELECT * FROM employees LIMIT 10;
```

WHAT THIS QUERY DOES

With no `ORDER BY`, 'first 10' really means 'whatever 10 rows the database happens to return first' — in practice this is usually insertion order, but that isn't officially guaranteed by SQL unless you explicitly sort. `LIMIT` alone is fine for quick previews, but for anything where order matters, pair it with `ORDER BY`.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bo
1	Aarav Sharma	28	M	75,000	IT	Senior Developer	*NULL*	2020-01-15	Mumbai	10
2	Priya Patel	32	F	65,000	HR	HR Manager	1	2019-03-20	Delhi	8,0
3	Rahul Singh	45	M	85,000	Finance	Finance Head	1	2018-07-10	Mumbai	15
4		29	F	72,000	IT	Developer	1	2021-02-01	Bangalore	9,0

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bo
	Neha Gupta									
5	Vikram Reddy	38	M	90,000	IT	Team Lead	1	2017-11-25	Delhi	12
6	Ananya Menon	27	F	60,000	HR	HR Executive	2	2022-06-15	Mumbai	5,0
7	Karthik Nair	51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	20
8	Meera Krishnan	33	F	68,000	Marketing	Marketing Lead	5	2020-08-12	Delhi	7,0
9	Arjun Reddy	42	M	82,000	IT	Senior Developer	5	2016-12-01	Mumbai	11
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4,0

10 rows returned.

Q22. Employees 11-20 EASY

```
SELECT * FROM employees LIMIT 10 OFFSET 10;
```

WHAT THIS QUERY DOES

OFFSET skips a number of rows before starting to return results. `LIMIT 10 OFFSET 10` means 'skip the first 10, then give me the next 10' — this is exactly the mechanism behind pagination (page 2 of search results, etc). Since our table only has 15 employees, this actually returns just 5 rows (11 through 15).

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bor
11	Rajesh Kumar	39	M	70,000	Finance	Financial Analyst	3	2019-06-10	Chennai	6,0
12	Deepa Iyer	34	F	64,000	HR	HR Specialist	2	2020-11-15	Hyderabad	5,5
13	Suresh Babu	47	M	88,000	IT	Solution Architect	5	2014-03-01	Chennai	18,0
14	Kavya Nair	31	F	67,000	Finance	Accountant	3	2021-08-20	Hyderabad	5,0
15	Ravi Teja	44	M	91,000	IT	Technical Manager	1	2016-07-15	Mumbai	16,0

5 rows returned.

Q23. Highest salary EASY

```
SELECT * FROM employees ORDER BY salary DESC LIMIT 1;
```

WHAT THIS QUERY DOES

The 'sort then slice' pattern from Q14, taken to its extreme: sort by salary descending, then keep only the very first row. This is a common (if slightly informal) way to find a single maximum row along with all its other column values — compare with `MAX(salary)` in Level 2, which gives you just the number, not the whole row.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bon
7	Karthik Nair	51	M	95,000	IT	Architect	1	2015-09-30	Bangalore	20,0

1 row returned.

Q24. Lowest salary EASY

```
SELECT * FROM employees ORDER BY salary ASC LIMIT 1;
```

WHAT THIS QUERY DOES

Mirror image of Q23 — sort ascending instead of descending, then take the first row, giving you the single lowest-paid employee's full record.

RESULT ON OUR SAMPLE DATA

emp_id	name	age	gender	salary	department	designation	manager_id	hire_date	city	bonu
10	Sneha Patel	26	F	58,000	Marketing	Marketing Executive	8	2023-01-20	Bangalore	4,000

1 row returned.

Q25. Second highest salary MEDIUM

```
SELECT MAX(salary) FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

WHAT THIS QUERY DOES

This is your first **subquery** — a query nested inside another one. The inner query `(SELECT MAX(salary) FROM employees)` runs first and finds the single highest salary (95,000). The outer query then asks for the maximum salary *among employees earning less than that* — effectively finding 'the biggest number that

isn't the biggest number,' i.e. the second-highest salary. This trick generalizes to 3rd-highest, 4th-highest, etc. by nesting further (see Q112).

RESULT ON OUR SAMPLE DATA

MAX(salary)
91,000

1 row returned.